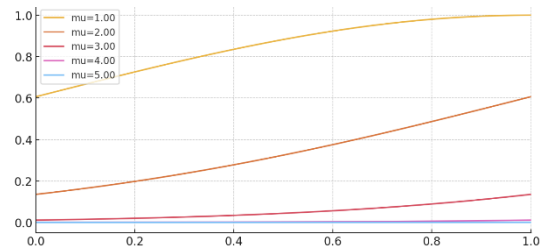
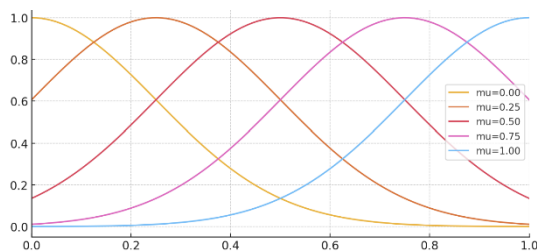


EE214 Assignment 1 Report

학번 비공개 SimHyeonseong

[Approach and Solution Outline]

First, since there is no function to fit or evaluate data in part 2 like 'polynomial(X, y, degree)' or 'gaussian(X, y, degree, sigma)', I defined several functions to implement training and testing repeatedly many times. While writing the codes, I asked ChatGPT how to get high performance. It recommended me to use L2 regularization for both basis (polynomial and Gaussian), and to set μ_j (average) and σ (standard deviation) as $\min(x) + \frac{\max(x) - \min(x)}{\text{degree} - 1} j$ for $j = 0, 1, \dots, \text{degree} - 1$ and $\frac{\max(x) - \min(x)}{\text{degree} - 1}$ each. According to the assignment instruction file, it says to use a linear regression model, not a Ridge. So, I accepted its recommendation only for modifying μ_j and σ because the original $\mu_j = j$ and $\sigma = 1$ make the output values of Gaussian basis functions too small so that the model can't learn properly. Since the input values of Gaussian basis are standardized, the scaler limits the values of x mostly in section $(-1, 1)$. Then, when $\mu_5 = 5$ and $x = 0$, the output value of Gaussian basis function is $e^{-\frac{(-5)^2}{1}} = e^{-25} \approx 0$ so it can ruin training. More specifically, when $\mu_j = j$ and $\sigma = 1$, the graphs of Gaussian basis look like the figure on the right side for $x \in [0, 1]$. And when $\mu_j = \min(x) + \frac{\max(x) - \min(x)}{\text{degree} - 1} j$ and $\sigma = \frac{\max(x) - \min(x)}{\text{degree} - 1}$, the graphs look



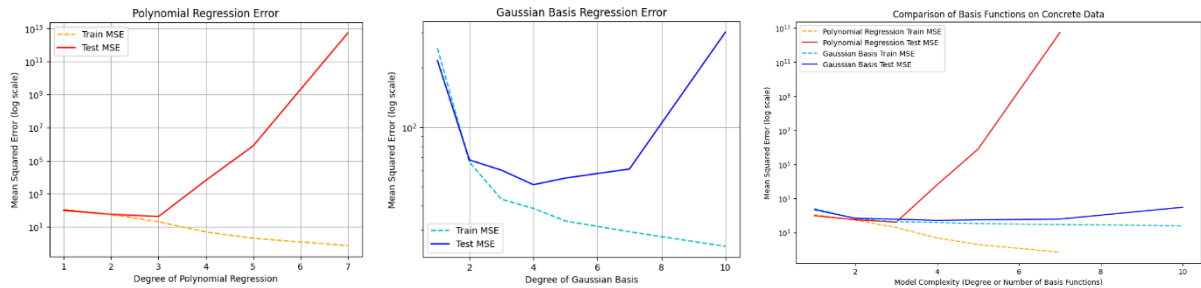
like the figure on the left side. I implemented the function 'gaussian' which makes input matrix X into X_{Gauss} using following method. If the shape of X is $(n_{\text{samples}}, n_{\text{features}})$, then the shape of matrix X_{Gauss} after applying Gaussian basis is $(n_{\text{samples}}, n_{\text{features}} \times \text{degree} + 1)$. It means that X_{Gauss} has all-1 vector for bias term in the 1st column, $(n_{\text{samples}}, \text{degree})$ shaped matrix transformed by Gaussian basis from the 1st feature vector of X in the columns from 2nd to $(\text{degree} + 1)^{\text{th}}$, and so on. Although there is another form of Gaussian basis which also considers combined terms of diverse features of X , it is too complex to implement, so my Gaussian basis treats only a single feature at a time.

I tried all degrees from 1 to 15 on both models. Identifying what degree shows good illustration of the error's trend, I chose the right ones to show the convexity of test error graph. For polynomial regression, I ran it with 6 degrees [1, 2, 3, 4, 5, 7] since it shows extremely high errors for both degree=6 and degree=7 so I just removed 6 from the list. For the same reason, I ran Gaussian regression with 7 degrees [1, 2, 3, 4, 5, 7, 10]. Also, I visualized train/test MSE vs. complexity(degree) curves of each model and curves of both models on the same plot for comparison.

In addition to the contents in the instruction file, I did other work additionally: visualizing $(\text{bias})^2$, variance, and total error graphs of models. Using 100 models trained from noisy training data, I

calculated them of each model. Plus, to maintain consistency of outputs, I fixed `np.random.seed(0)` before making noise using `np.random.normal` and set the parameter `random_state` of function `train_test_split` explicitly.

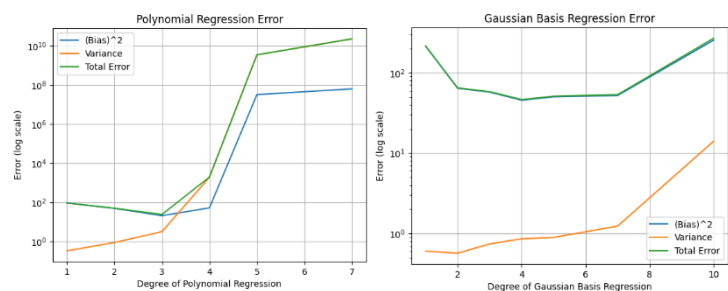
[Results and Analysis] – Answers for Analysis Questions



(a) According to three graphs above, polynomial regression at degree 3 provided the best minimum error (40.27) among all combinations, which is smaller than the minimum error (50.90) of Gaussian regression at degree 4. However, as degree increases beyond 3, test MSE of polynomial model skyrocketed, whereas Gaussian model shows stable and relatively smaller error overall at high degrees. This is because polynomial regression at degree 3 is flexible enough to capture the overall features of data 'Concrete_Data.xls' without overfitting. Also, Gaussian regression shows more stable generalization compared to polynomial one since Gaussian basis ensures locality, that is, if x is far from μ_j , then the basis affects only small region of the input and it is less sensitive.

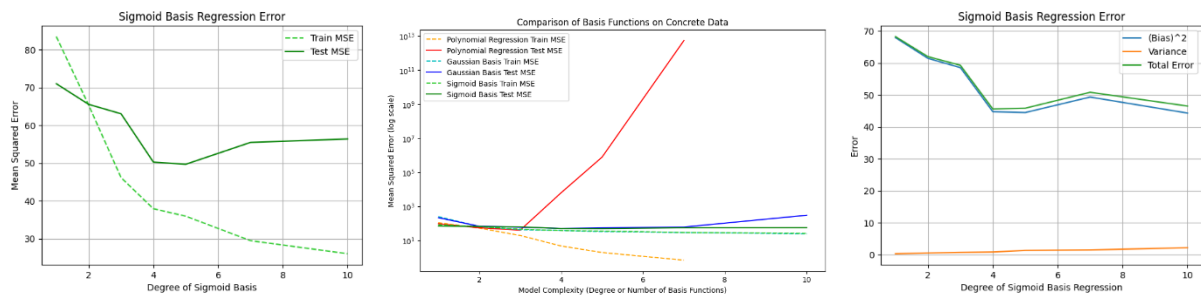
(b) In both models, as degrees increase, training MSEs gradually decrease. However, test MSEs decrease until the best degree (3 and 4 each) and go higher after it showing convexity. Some examples of underfitting are polynomial at degree 1 and Gaussian at degree 1. They provide high errors for both train and test. Some examples of overfitting are polynomial at degree 7 and Gaussian at degree 10. They provide very high test errors and much lower training errors.

(c) Using 100 models trained from noisy training data, I plotted $(\text{bias})^2$, variance graphs for both. Since we don't know the ground-truth functions for 'Concrete_Data.xls', I considered `y_test` on `random_state=42` as ground-truth values for `X_test`. According to the graphs, as degrees increase, variances also increase. It means that models with high degrees are very sensitive to noise and overfit. At low degree (e.g. 1), both models are too simple to capture the features of data, so it shows high bias and low variance, indicating underfitting. At best degree (3 and 4 respectively), since they achieved great balance between bias and variance, they provide the best performance. At degrees from 1 to best ones, variance increases and bias decreases as complexity increases, so these show the bias-variance tradeoff to achieve best performance. In this experiment, I noticed that there are unexpected results which are different from theory. I have expected $(\text{bias})^2$ graphs to go down as degree increases



unlike variance, but they increase along with variance at high degrees. This might happen due to the following reasons. Models with high degrees generate matrices X_{poly} or X_{Gauss} , which have too many features, so they cause numerical instability and amplify small bias. Also, those extremely overfitting models try to fit noise in training data, increasing bias.

(d) Polynomial basis is simple to implement and understand. It is also flexible at low degrees. However, degree decides everything, that is, it is highly sensitive to and depends on degree choice. If we select an improper degree for the model, it will overfit extremely. Since Gaussian basis has locality over input data, it is more stable and robust against overfitting compared to polynomial basis, so it is good for complex data. But it requires careful tuning of μ_j and σ , and may need large numbers of basis functions to capture the features of complex data. Thus, while polynomial basis is suitable for simple problems, Gaussian basis is suitable for complex real-world problems.



(e) I applied sigmoid basis: $\phi(x) = \frac{1}{1 + e^{-\frac{x - \mu_j}{\sigma}}}$ where μ_j and σ have the same equations for Gaussian.

One of the characteristics of it is that it limits the value in section (0,1). I also applied L2 regularization on sigmoid regression with $\alpha=1e-6$. According to the graphs, it shows the lowest error at degrees 4 and 5. As degree increases, variance increases, and bias tends to decrease although there is a depression at the midpoints where the best degrees (4 and 5) are located. With low degree, the model shows high bias and low variance, resulting in underfitting. If degree is high, it shows high variance, so it becomes sensitive to noise and overfit. Considering bias-variance trade off, we have to choose the best degrees to balance between bias and variance. Due to the limiting characteristics of sigmoid basis, it can capture saturating behavior in real-world problems such as biological growth, or chemical reactions. It is more stable and localized like Gaussian basis compared to polynomial basis. However, it requires tuning of μ_j and σ , and may need large numbers of basis functions to capture the features of complex data.

[Use of AI]

I have mainly used ChatGPT, and sometimes used Gemini in Colab or code autocompletion in Colab. I used AI tools to get explanations about concept of eigen decomposition from covariance matrix, PCA via power iteration in part 1, and how to get high performance in polynomial and Gaussian basis regression, and another basis function other than polynomial and Gaussian basis. I also used them to generate snippets of codes about imputing missing values from data, method to plot colored-dotted graph, how to generate matrix X_{Gauss} which is transformed from X by Gaussian basis in python, and generating noisy training data to apply in 100 models to identify

bias-variance tradeoff. Additionally, I asked them whether the graph figures I got are somewhat reasonable, or wrong and need to be modified. For imputing missing values, I just copied them because they are short: just 2 lines. But in repeating for $n_{\text{models}} (=100)$ to get bias and variance graph and in defining functions to create X_{Gauss} and X_{sigmoid} , I modified them from the codes AI generated. For example, in repeating 100 times, ChatGPT just provide me general form of the codes, so I modified them to fit the variables on the functions I defined. And in defining two functions, 'create_gaussian_design_matrix' and 'create_full_gaussian_design_matrix', I introduced new parameter 'include_bias' and set the default value of given parameter 'sigma' as 'auto' to treat it inside the functions when sigma is not provided as specific number. The parameter 'include_bias' is true only at first implementation and false otherwise. It means that the function generates vector for bias term only once in X_{Gauss} . The 1st feature vector of X is transformed into $(n_{\text{samples}}, \text{degree} + 1)$ shaped matrix including bias term, and the other feature vectors are transformed into just $(n_{\text{samples}}, \text{degree})$ shaped matrix without bias. There is an instance where AI didn't give me proper answer. I asked ChatGPT to check whether the $(\text{bias})^2$ and variance graph I generated is reasonable or not. As I explained in the part **[Analysis Questions – (c)]**, bias increases at high degrees unlike the theory, which says bias decreases as complexity increases. However, ChatGPT answered that it is perfect, so I changed my question into this: "According to the theory, bias-variance tradeoff, bias should decrease as degree increases, but it increases after the best degree in my case. Why does this happen?". I wrote 4 citations in "Assignment1_Part2.ipynb" as follows:

```
# [Citation] The codes in the loop "for i in range(n_models)" are made from snippet provided by ChatGPT.
for degree in degrees:
    predictions = [] # To store predictions from each model
    # Train n_models models on different training samples
    for i in range(n_models):
        # Generate a new training set for each model
        X_train_var, y_train_var, X_test, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=1000+i)
        y_train_var += np.random.normal(0, 0.1*np.std(y_train_var), len(y_train_var))
        y_train_pred, y_test_pred, train_mse, test_mse = polynomial(X_train_var, X_test, y_train_var, y_test, degree)
        predictions.append(y_test_pred)

#-----Handle missing values if present-----
# Separate columns into two groups of continuous and integer types
continuous_cols = [col for col in data.columns if data[col].dtype == 'float64']
integer_cols = [col for col in data.columns if data[col].dtype == 'int64']
# [Citation] Above two lines to separate columns into two groups are generated by Colab code autocompletion.

# [Citation] The codes imputing missing values are generated by Gemini in Colab.
# Impute missing values in columns of continuous type with the mean
imputer_continuous = SimpleImputer(strategy='mean')
X = imputer_continuous.fit_transform(X)

# Impute missing values in columns of integer type with the most frequent
imputer_integer = SimpleImputer(strategy='most_frequent')
X = imputer_integer.fit_transform(X)
```

```
# [Citation] More than half of the structures of two functions below, 'create_gaussian_design_matrix'
# and 'create_full_gaussian_design_matrix', are generated by ChatGPT.
def create_gaussian_design_matrix(x, degree, sigma='auto', include_bias=True):
    """
    Create a design matrix using Gaussian basis functions.
    Args:
        x: numpy array of shape (n_samples,)
        degree: number of Gaussian basis functions
        sigma: width of the Gaussian functions
    Returns:
        X: design matrix with shape (n_samples, degree) or (n_samples, degree + 1) including bias term
    """
    #-----Blank: Fill in Gaussian basis transformation-----
    # TODO: Fill in the loop to compute Gaussian basis functions
    n_samples = x.shape[0]
    bias_term = 1 if include_bias else 0
    X = np.ones((n_samples, degree + bias_term))
```